

```
$ telnet localhost 2947

Trying ::1...
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^)'.
{"class": "VERSION", "release": "2.96-
dev", "rev": "2011-03-
15T03:05:33", "proto_major": 3, "proto_
minor": 4}
```

Now to see data from the receiver, enter the command:

```
?WATCH={"enable":true,"json":true}
```

The data appears like this:

```
{ "class": "TPV", "tag":
"RMC", "device": "/dev/
ttyUSB0", "mode": 3, "time": "2013-05-
07T17:10:16.000Z", "ept": 0.005, "lat": 24.08
8710000, "lon": 82.648190000, "alt": 302.20
0, "epx": 2.054, "epy": 3.105, "epv": 7.705, "tr
ack": 344.0400, "speed": 0.010, "climb":
0.100, "eps": 6.21}
```

```
{ "class": "TPV", "tag":
"RMC", "device": "/dev/
ttyUSB0", "mode": 3, "time": "2013-05-
07T17:10:17.000Z", "ept": 0.005, "lat": 24.08
8710000, "lon": 82.648186667, "alt": 302.30
0, "epx": 2.054, "epy": 3.105, "epv": 7.705, "tr
ack": 344.0400, "speed": 0.005, "climb": 0.10
0, "ens": 6.21 }
```

Now here is a Python script which exploits every GPS data in your own way. The script has some issue related to the absolute path. Therefore open it in /usr/sbin directory of your Raspberry Pi where the 'gpsd' program file is located.

Create the file:

```
$ sudo nano gpsdata.py
```

Paste the code below in that file:

```
import os
```

```

from gps import *
from time import *
import time
import threading

gpsd = None #setting the global
variable
os.system('clear') #clear the
terminal (optional)

class GpsPoller(threading.Thread):
    def __init__(self):
        threading.Thread.__init__(self)
        global gpsd #bring it in scope
        gpsd = gps(mode=WATCH_ENABLE)
#starting the stream of info
        self.current_value = None
        self.running = True #setting the
            thread running to true

    def run(self):
        global gpsd
        while gpsd.running:
            gpsd.next() #this will continue
to loop and grab EACH set of gpsd
info to clear the buffer

if __name__ == '__main__':
    gpsp = GpsPoller() # create the
        thread

    try:
        gpsp.start() # start it up
        while True:
            #It may take a second or two to
                get good data
            #print gpsd.fix.latitude,',
                ',gpsd.fix.longitude,' Time: ',gpsd.
                utc

```

```

os.system('clear')
print 'Raspberry Pi GPS reading'
print '-----'
print 'latitude ', gpsd.fix.latitude
print 'longitude ', gpsd.fix.longitude
print 'time utc ', gpsd.utc, ' + ',
    gpsd.fix.time
print 'altitude (m) ', gpsd.fix.
    altitude
print 'eps ', gpsd.fix.eps
print 'epx ', gpsd.fix.epx
print 'epv ', gpsd.fix.epv
print 'ept ', gpsd.fix.ept
print 'speed (m/s) ', gpsd.fix.speed
print 'climb ', gpsd.fix.climb
print 'track ', gpsd.fix.track
print 'mode ', gpsd.fix.mode
    print
print 'sats ', gpsd.satellites

    time.sleep(5) #set to whatever

except (KeyboardInterrupt,
SystemExit): #when you press ctrl+c
    print "\nKilling Thread..."

gpsp.running = False
gpsp.join() # wait for the thread
    to finish what it's doing
    print "Done.\nExiting."
```

Please take care that the indentations of the pasted code are the same as mentioned above as Python is sensitive to it.

Now run the code by using the command below and the GPS data appears as shown in Fig. 7.

```
$ python gpsdata.py
```

To set our Raspberry Pi time synchronised with GPS time NMEA satellite protocol, we can install 'ntp' program as:

```
$sudo apt-get install ntp
```

Now run the command below and the time of the Raspberry Pi will now be synchronised with Pacific Time standard.

```
$sudo service ntp
restart
```

The author is an avid user of open source software. Professionally, he is a thermal power expert and works as an additional general manager at NTPC Limited



Parameter	Value
Time:	2013.05-04T14:48:58.000Status: JD FET (901 secs)
Latitude:	24.088781 N EPR: 8.963 m
Longitude:	82.648206 E EPR: 7.463 m
Altitude:	269.60 m EPR: 23.818 m
Speed:	0.018 kph EPR: 17.939 m
Climb:	0.600 kph EPR: n/a
Track:	102.120900 EPR: n/a

Fig. 6: GPS data in xgps

```
Raspberry Pi 3B+ reading
*****
Latitude 24.089783067
Longitude 82.646173333
Speed vbc 703.550 -00T01:20:57.000Z + 1367976537.0
Altitude (m) 294.2
gpx 4.08
epx 1.009
vpx 7.30
vay 0.004
speed (m/s) 0.005
GLIMB 0.0
track 153.04
mode 3
$PWR: [PWR]: A1: 63 Az: 295 S5: 32 Used:
M: 38 E: 51 Az: 52 S5: 29 Used: y, PWR: 21 E:
48 Az: 52 Used: y, PWR: 0 E: 20 Az: 251
Used: y, PWR: 0 E: 20 Az: 292 S5: 32 Used: y,
PWR: 1 E: 7 Az: 123 S5: 35 Used: y, PWR: 12 E: 1
```

Fig. 7: GPS reading in command window